

SAP BTP Trial API Collection (Bruno) Playbook

How to run this playbook: In an AI coding assistant (e.g. Claude Code), type:

```
run this playbook btp-trial-api-playbook.md
```

The AI coding assistant reads the playbook, discovers your account values, creates the required service instances/bindings via the `btp` CLI, and **writes all `bruno.json`, `.bru` request, and environment files directly** — filling in your account's values. Secrets live in a single `.env` file at the collection root (git-ignored); the environment files reference them via `{{process.env.*}}`, so **no secret ever sits in a committed file**. A few steps (the IAS application) are **manual in the IAS Admin Console** — these are clearly marked. Placeholders use `<UPPER_SNAKE_CASE>`.

Before you start: read `btp-trial-api-prerequisites.md` — it covers the trial account, tools, and the safety disclaimer (AI can err; keep a human in the loop; never use on productive accounts).

PDF vs. Markdown — which copy to use: A PDF export of this playbook is for **humans to read and share**. An **AI coding assistant must execute the Markdown (`.md`) version, never the PDF**. Reason: PDF text extraction reflows content — it rewraps long commands and URLs, collapses the wide 35-row request table, and can drop or mangle exact whitespace and tokens like `{{process.env.*}}`. Those details are load-bearing here (a reflowed URL or split token produces a wrong request), so only the Markdown preserves them faithfully. If you only have the PDF, convert it back to Markdown before running it.

This playbook rebuilds a known-good Bruno API-testing collection from an **empty folder**, pointed at **any** SAP BTP **trial** account. End state: **9 folders / 35 requests / 6 environments**, all returning `200` (with two documented exceptions — see Gotchas #5 and the IAS browser flow).

The collection covers the BTP Account Administration REST APIs: Accounts, Entitlements, Provisioning, Events, SaaS subscriptions, Service Manager, XSUAA (users/roles), Usage Reporting, and IAS OAuth2.

System Details

Fill these in during Step 1. Placeholders below are used throughout the playbook.

Property	Placeholder	Example / How to find
Global account subdomain	<GLOBAL_ACCOUNT>	<code>btp login target</code> , e.g. <code>a1b2c3d4trial-ga</code>
Subaccount subdomain	<SUBACCOUNT_SUBDOMAIN>	from <code>Subaccount-list</code> , e.g. <code>a1b2c3d4trial</code>
Subaccount ID (GUID)	<SUBACCOUNT_ID>	from <code>Subaccount-list guid</code>
Central region	<CENTRAL_REGION>	almost always <code>eu10</code> (see Gotcha #1)
Subaccount region	<SUBACCOUNT_REGION>	from <code>Subaccount-list region</code> , e.g. <code>ap21</code>
Collection name	<COLLECTION_NAME>	your choice, e.g. <code>btp-trial-api</code>
IAS tenant host	<IAS_HOST>	from <code>AvailableIDP-list host</code> , e.g. <code>abcd1234.trial-accounts.ondemand.com</code>

Credentials (produced by Steps 2–3, written to `.env` in Step 6a)

Source	client_id placeholder	client_secret placeholder
<code>cis-central binding</code>	<CIS_CENTRAL_CLIENT_ID>	<CIS_CENTRAL_CLIENT_SECRET>
<code>cis-local binding</code>	<CIS_LOCAL_CLIENT_ID>	<CIS_LOCAL_CLIENT_SECRET>
<code>uas-reporting binding</code>	<UAS_CLIENT_ID>	<UAS_CLIENT_SECRET>
<code>sm-audit binding</code>	<SM_CLIENT_ID>	<SM_CLIENT_SECRET>
<code>xsuaa-api-cred</code>	<XSUAA_CLIENT_ID>	<XSUAA_CLIENT_SECRET>
IAS app <code>bruno-test</code>	<IAS_CLIENT_ID>	<IAS_CLIENT_SECRET>

Status

Task	Status
Step 1 — Log in & discover subaccount	"
Step 2 — Create instances, bindings & credentials	"
Step 3 — Create IAS application (manual)	"
Step 4 — Scaffold collection	"
Step 5 — Write request files	"

Task	Status
Step 6 — Write environment files	"
Step 7 — Verify	"

Collection Layout

<COLLECTION_NAME>/	
├ bruno.json	
├ .env	secrets only (git-ignored – Gotcha #7)
├ .gitignore	→ .env
├ environments/	(6 env files – reference {{process.env.*}} – Step 6)
│ └ trial-global.bru	CIS central – <CENTRAL_REGION>
│ └ trial-local.bru	CIS local – <SUBACCOUNT_REGION>
│ └ trial-xsuaa.bru	XSUA API – <SUBACCOUNT_REGION>
│ └ trial-uas.bru	Usage report – <CENTRAL_REGION>
│ └ trial-sm.bru	Service Mgr – <SUBACCOUNT_REGION>
│ └ trial-ias.bru	IAS OAuth2
├ auth/	
│ └ get-token.bru	shared – writes access_token into active env
├ global-accounts/	→ trial-global
├ global-entitlements/	→ trial-global
├ local-provisioning/	→ trial-local
├ local-events/	→ trial-local
├ xsuaa/	→ trial-xsuaa
├ usage/	→ trial-uas (2 endpoints return 500 in trial)
├ service-manager/	→ trial-sm
└ ias/	→ trial-ias (auth-code needs Bruno Desktop)

Environment → capability matrix

	trial-global	trial-local	trial-xsuaa	trial-uas	trial-sm	trial-ias
Base credential	cis-central	cis-local	xsuaa-api-cred	uas-reporting	sm-audit	IAS app
Global Account / Entitlements	✓					
Provisioning / Events / SaaS		✓				
Users / Roles / IdPs			✓			
Usage Reporting				✓		
Service Manager					✓	
IAS OAuth2						✓

Playbook

Step 1 — Log in & Discover Subaccount

Log in to the BTP CLI (SSO opens a browser; your password never touches the terminal):

```
btp login --url https://cli.btp.cloud.sap --subdomain <GLOBAL_ACCOUNT> --sso
```

Discover the subaccount's ID, region, and subdomain via the `btp` CLI:

```
btp list accounts/subaccount
```

→ note `guid` (→ `<SUBACCOUNT_ID>`), `region` (→ `<SUBACCOUNT_REGION>`), `subdomain` (→ `<SUBACCOUNT_SUBDOMAIN>`).

Fill the System Details table. This step is read-only — no confirmation needed.

Step 2 — Create Service Instances, Bindings & Credentials (btp CLI)

Always ask the user for confirmation before executing this step. It creates service instances and bindings in the subaccount.

For each of 2a–2e: create the instance, create a binding, then read the credentials with `btptool get services/binding --take credentials.uaa.clientid` and `credentials.uaa.clientsecret` into the matching placeholders.

Shell quoting: `clientid` / `clientsecret` contain `!`, `|`, `$` — always wrap them in **single quotes** in shell commands.

Step 2a — cis-central (→ trial-global)

```
btptool create services/instance --subaccount <SUBACCOUNT_ID> \
  --offering-name cis --plan-name central --name cis-central \
  --parameters '{"grantType":"clientCredentials"}'
btptool create services/binding --name cis-central-binding --instance-name cis-central --subaccount <SUBACCOUNT_ID>
btptool get services/binding --name cis-central-binding --subaccount <SUBACCOUNT_ID>
```

Step 2b — cis-local (→ trial-local)

Important (Gotcha #2): the `--parameters '{"grantType":"clientCredentials"}'` is **mandatory**. Without it the token only carries the `uaa.resource` scope and every provisioning/events call fails with **403 insufficient_scope**.

```
btptool create services/instance --subaccount <SUBACCOUNT_ID> \
  --offering-name cis --plan-name local --name cis-local \
  --parameters '{"grantType":"clientCredentials"}'
btptool create services/binding --name cis-local-binding --instance-name cis-local --subaccount <SUBACCOUNT_ID>
btptool get services/binding --name cis-local-binding --subaccount <SUBACCOUNT_ID>
```

Step 2c — uas-reporting (→ trial-uas)

```
btptool create services/instance --subaccount <SUBACCOUNT_ID> \
  --offering-name uas --plan-name reporting-ga-admin --name uas-reporting
btptool create services/binding --name uas-reporting-binding --instance-name uas-reporting --subaccount <SUBACCOUNT_ID>
btptool get services/binding --name uas-reporting-binding --subaccount <SUBACCOUNT_ID>
```

Step 2d — service-manager (→ trial-sm)

```
btptool create services/instance --subaccount <SUBACCOUNT_ID> \
  --offering-name service-manager --plan-name subaccount-audit --name sm-audit
btptool create services/binding --name sm-audit-binding --instance-name sm-audit --subaccount <SUBACCOUNT_ID>
btptool get services/binding --name sm-audit-binding --subaccount <SUBACCOUNT_ID>
```

Step 2e — xsuaa api-credential (→ trial-xsuaa)

The XSUAA REST APIs use an **API credential**, not a service binding:

```
btptool create security/api-credential --name xsuaa-api-cred --subaccount <SUBACCOUNT_ID>
```

→ note the `Client ID` and `Client Secret` from the output.

Note: for a read-only collection, add `--read-only` — the credential then cannot perform write operations.

Step 3 — Create the IAS Application (manual)

Note: This is not a CLI step — it is done in the IAS Admin Console. Two ways to do it:

- **Manually** in the browser (default, most reliable), or
- **Semi-automated** with a **browser automation tool** (e.g. Chrome DevTools MCP, Playwright MCP) that lets an AI coding assistant drive the Admin Console UI. Caveats: the AI coding assistant cannot log you in (interactive SSO / 2FA stays with you), UI automation is brittle (session timeouts, layout changes), and the client secret is shown only once — so **you** still capture it. Confirm each click before it runs.

Find `<IAS_HOST>` — the IAS tenant trusted by the subaccount. Run:

```
btptool list security/trust --subaccount <SUBACCOUNT_ID>
```

→ look for the entry with **Origin** `sap.custom` — its **Identity Provider** value is `<IAS_HOST>` (e.g. `abcd1234.trial-accounts.ondemand.com`). Alternatively find it in the BTP Cockpit: Subaccount → **Security** → **Trust Configuration** → the custom entry.

In the IAS Admin Console at `https://<IAS_HOST>/admin/` :

1. **Applications & Resources** → **Applications** → **Create** — Display Name `bruno-test`, Protocol Type `OpenID Connect`.
2. **Trust** → **OpenID Connect Configuration** — Name `bruno-test`, Redirect URI `http://localhost:8686/callback`, enable Grant Type **Authorization Code**. Save.
3. **Trust** → **Client Authentication** → **Add** (under Secrets) — Description `bruno-test-secret`, API Access: **OpenID + Application Users**. The secret is **shown only once** — **record it immediately** into `<IAS_CLIENT_ID>` / `<IAS_CLIENT_SECRET>`.

Important (Gotcha #4): In Bruno Desktop, set **Preferences** → **Use system browser for OAuth** → **OFF**. Otherwise the callback to `localhost:8686` cannot be intercepted and the login fails.

Step 4 — Scaffold the Collection

Always ask the user for confirmation before executing this step. It creates files in the target folder.

In the empty target folder, write `bruno.json` :

```
{
  "version": "1",
  "name": "<COLLECTION_NAME>",
  "type": "collection",
  "ignore": ["node_modules", ".git"]
}
```

Write `.gitignore` (protects the secrets file — see Gotcha #7):

```
.env
node_modules/
```

Create the 8 request folders + `auth/` + `environments/` (see Collection Layout). The `.env` file itself is written in Step 6, once the credentials from Steps 2–3 are known.

Note: Do **not** create a file named `json` (no extension) — that is a leftover `bru run` results artifact in the reference collection and would leak a token.

Step 5 — Write Request Files

Note (Gotcha #3): the env files reference `client_id` / `client_secret` as **normal vars** pulling from `{{process.env.*}}` (defined in `.env`) — not Bruno `vars:secret`. Bruno deletes empty Secrets, and the `bru` CLI cannot read Secrets. See Security for the desktop-only alternative.

.bru anatomy: a `meta { }` block (name, type, seq), a method block (`get / post` with `url`, `body`, `auth`), and an auth block matching the `auth:` value. Only `auth/get-token` has a `script:post-response`.

Every request uses one of **5 patterns**:

Pattern A — Bearer GET (most requests):

```
meta {
  name: <name>
  type: http
  seq: 1
}
get {
  url: <URL with {{vars}}>
  body: none
  auth: bearer
}
auth:bearer {
  token: {{access_token}}
}
```

Pattern B — Token POST (only `auth/get-token`, the only file with a script):

```

meta {
  name: get-token
  type: http
  seq: 1
}
post {
  url: {{uaa_url}}/oauth/token
  body: formUrlEncoded
  auth: basic
}
auth:basic {
  username: {{client_id}}
  password: {{client_secret}}
}
body:form-urlencoded {
  grant_type: client_credentials
}
script:post-response {
  bru.setEnvVar("access_token", res.getBody().access_token);
}

```

Pattern C — No-auth GET (body: none, auth: none, no auth block): jwks, openid-configuration, token-keys.

Pattern D — OAuth2 Authorization Code (IAS browser flow — get-token-auth-code and get-userinfo):

```

meta {
  name: <name>
  type: http
  seq: 1
}
get {
  url: {{ias_url}}/oauth2/userinfo
  body: none
  auth: oauth2
}
auth:oauth2 {
  grant_type: authorization_code
  callback_url: http://localhost:8686/callback
  authorization_url: {{ias_url}}/oauth2/authorize
  access_token_url: {{ias_url}}/oauth2/token
  refresh_token_url:
  client_id: {{client_id}}
  client_secret: {{client_secret}}
  scope: openid
  state:
  pkce: false
  credentials_placement: header
  credentials_id: credentials
  token_source: access_token
  token_placement: header
  token_header_prefix: Bearer
  auto_fetch_token: true
  auto_refresh_token: false
}

```

Pattern E — Basic-auth form POST (only ias/introspect-token):

```

post {
  url: {{ias_url}}/oauth2/introspect
  body: formUrlEncoded
  auth: basic
}
auth:basic {
  username: {{client_id}}
  password: {{client_secret}}
}
body:form-urlencoded {
  token: {{access_token}}
}

```

All 35 requests (create each with the indicated pattern; substitute `{{vars}}` verbatim — they resolve from the environment):

Folder	File	Method	URL	Pattern	Env	Notes
auth	get-token	POST	<code>{{uaa_url}}/oauth/token</code>	B	any	shared; writes <code>accessToken</code>
global-accounts	get-global-account	GET	<code>{{btp_accounts_api}}/accounts/v1/globalAccount</code>	A	trial-global	
global-accounts	get-global-account-hierarchy	GET	<code>{{btp_accounts_api}}/accounts/v1/globalAccount?expand=true</code>	A	trial-global	
global-accounts	get-subaccount-details	GET	<code>{{btp_accounts_api}}/accounts/v1/subaccounts/{{subaccount_id}}</code>	A	trial-global	
global-accounts	list-subaccounts	GET	<code>{{btp_accounts_api}}/accounts/v1/subaccounts</code>	A	trial-global	
global-entitlements	get-entitlements-global-account	GET	<code>{{btp_entitlements_api}}/entitlements/v1/assignments</code>	A	trial-global	
global-entitlements	get-entitlements-subaccount	GET	<code>{{btp_entitlements_api}}/entitlements/v1/assignments?subaccountGUID={{subaccount_id}}</code>	A	trial-global	
local-provisioning	get-environment-instances	GET	<code>{{btp_provisioning_api}}/provisioning/v1/environments</code>	A	trial-local	bearer (fix: reference <code>auth: none</code>)
local-provisioning	get-environment-instance-details	GET	<code>{{btp_provisioning_api}}/provisioning/v1/environments/{{environment_instance_id}}</code>	A	trial-local	parameterized; fill <code>environment_instances</code> from list
local-provisioning	get-subscriptions	GET	<code>{{btp_saas_registry_api}}/saas-manager/v1/applications</code>	A	trial-local	
local-provisioning	get-subscription-details	GET	<code>{{btp_saas_registry_api}}/saas-manager/v1/applications/{{app_name}}?planName={{app_plan}}</code>	A	trial-local	parameterized
local-events	get-events	GET	<code>{{btp_events_api}}/cloud-management/v1/events</code>	A	trial-local	
local-events	get-events-subaccount	GET	<code>{{btp_events_api}}/cloud-management/v1/events?entityType=Subaccount</code>	A	trial-local	
local-events	get-event-types	GET	<code>{{btp_events_api}}/cloud-management/v1/events/types</code>	A	trial-local	
xsuaa	get-role-collections	GET	<code>{{xsuaa_api_url}}/sap/rest/authorization/v2/rolecollections</code>	A	trial-xsuaa	
xsuaa	get-roles	GET	<code>{{xsuaa_api_url}}/sap/rest/authorization/v2/roles</code>	A	trial-xsuaa	
xsuaa	get-users	GET	<code>{{xsuaa_api_url}}/Users</code>	A	trial-xsuaa	SCIM
xsuaa	get-apps	GET	<code>{{xsuaa_api_url}}/sap/rest/authorization/v2/apps</code>	A	trial-xsuaa	
xsuaa	get-identity-providers	GET	<code>{{xsuaa_api_url}}/sap/rest/identity-providers</code>	A	trial-xsuaa	no <code>v2</code> in path
xsuaa	get-security-settings	GET	<code>{{xsuaa_api_url}}/sap/rest/authorization/v2/securitySettings</code>	A	trial-xsuaa	
xsuaa	get-token-keys	GET	<code>{{uaa_url}}/token_keys</code>	C	trial-xsuaa	no auth
usage	get-cloud-credits	GET	<code>{{uas_api}}/reports/v1/cloudCreditsDetails</code>	A	trial-uas	
usage	get-monthly-usage	GET	<code>{{uas_api}}/reports/v1/monthlyUsage?month={{month}}&year={{year}}</code>	A	trial-uas	500 in trial (Gotcha #1)
usage	get-subaccount-usage	GET	<code>{{uas_api}}/reports/v1/subaccountUsage?subaccountGUID={{subaccount_id}}&month={{month}}&year={{year}}</code>	A	trial-uas	500 in trial (Gotcha #2)

Folder	File	Method	URL	Pattern	Env	Notes
service-manager	get-service-instances	GET	{{sm_url}}/v1/service_instances	A	trial-sm	
service-manager	get-service-bindings	GET	{{sm_url}}/v1/service_bindings	A	trial-sm	
service-manager	get-service-offerings	GET	{{sm_url}}/v1/service_offerings	A	trial-sm	
service-manager	get-service-plans	GET	{{sm_url}}/v1/service_plans	A	trial-sm	
service-manager	get-platforms	GET	{{sm_url}}/v1/platforms	A	trial-sm	
service-manager	get-brokers	GET	{{sm_url}}/v1/service_brokers	A	trial-sm	
ias	get-token-auth-code	GET	{{ias_url}}/oauth2/userinfo	D	trial-ias	Bruno Desktop only (l
ias	get-userinfo	GET	{{ias_url}}/oauth2/userinfo	D	trial-ias	Bruno Desktop only (l
ias	introspect-token	POST	{{ias_url}}/oauth2/introspect	E	trial-ias	
ias	get-jwks	GET	{{ias_url}}/oauth2/certs	C	trial-ias	no auth
ias	get-openid-configuration	GET	{{ias_url}}/.well-known/openid-configuration	C	trial-ias	no auth

Step 6 — Write the .env File and Environment Files

Always ask the user for confirmation before executing this step. It writes the secrets file and the environment files.

Step 6a — Write .env (secrets only)

Write the six credential pairs from Steps 2–3 into `.env` at the collection root. This is the **only** file that holds secrets, and it is git-ignored (Step 4).

```
CIS_CENTRAL_CLIENT_ID=<CIS_CENTRAL_CLIENT_ID>
CIS_CENTRAL_CLIENT_SECRET=<CIS_CENTRAL_CLIENT_SECRET>
CIS_LOCAL_CLIENT_ID=<CIS_LOCAL_CLIENT_ID>
CIS_LOCAL_CLIENT_SECRET=<CIS_LOCAL_CLIENT_SECRET>
XSUAA_CLIENT_ID=<XSUAA_CLIENT_ID>
XSUAA_CLIENT_SECRET=<XSUAA_CLIENT_SECRET>
UAS_CLIENT_ID=<UAS_CLIENT_ID>
UAS_CLIENT_SECRET=<UAS_CLIENT_SECRET>
SM_CLIENT_ID=<SM_CLIENT_ID>
SM_CLIENT_SECRET=<SM_CLIENT_SECRET>
IAS_CLIENT_ID=<IAS_CLIENT_ID>
IAS_CLIENT_SECRET=<IAS_CLIENT_SECRET>
```

Note: `.env` values are **not** quoted, even though the BTP secrets contain `!`, `|`, `$`, `[`, `]`. Bruno reads them literally. Do not wrap them in quotes.

Step 6b — Write the environment files

Important (Gotcha #1 — region rule): Accounts, Entitlements, and Usage Reporting always run in the central region (`<CENTRAL_REGION>` , normally `eu10`) regardless of where the subaccount lives — these are central BTP services. **Provisioning, Events, SaaS, Service Manager, and XSUAA use the subaccount region (`<SUBACCOUNT_REGION>`).** The `uaa_url` for `global/uas` uses the **global-account** subdomain; for `local/xsuaa/sm` it uses the **subaccount** subdomain.

Write one `vars { }` block per environment. `client_id / client_secret` pull from `.env` via `{{process.env.*}}` — **no secret is written into these files.** `access_token` is present but **blank** in the five non-IAS envs (populated at runtime by `auth/get-token`); **omit** it from `trial-ias` (Bruno caches the IAS token internally).

trial-global

```
vars {
  btp_accounts_api: https://accounts-service.cfapps.<CENTRAL_REGION>.hana.ondemand.com
  btp_entitlements_api: https://entitlements-service.cfapps.<CENTRAL_REGION>.hana.ondemand.com
  btp_provisioning_api: https://provisioning-service.cfapps.<CENTRAL_REGION>.hana.ondemand.com
  btp_events_api: https://events-service.cfapps.<CENTRAL_REGION>.hana.ondemand.com
  btp_saas_registry_api: https://saas-manager.cfapps.<CENTRAL_REGION>.hana.ondemand.com
  uaa_url: https://<GLOBAL_ACCOUNT>.authentication.<CENTRAL_REGION>.hana.ondemand.com
  global_account: <GLOBAL_ACCOUNT>
  subaccount_id: <SUBACCOUNT_ID>
  client_id: {{process.env.CIS_CENTRAL_CLIENT_ID}}
  client_secret: {{process.env.CIS_CENTRAL_CLIENT_SECRET}}
  access_token:
}
```

trial-local (provisioning/events/saas on <SUBACCOUNT_REGION> ; accounts/entitlements on <CENTRAL_REGION>)

```
vars {
  btp_accounts_api: https://accounts-service.cfapps.<CENTRAL_REGION>.hana.ondemand.com
  btp_entitlements_api: https://entitlements-service.cfapps.<CENTRAL_REGION>.hana.ondemand.com
  btp_provisioning_api: https://provisioning-service.cfapps.<SUBACCOUNT_REGION>.hana.ondemand.com
  btp_events_api: https://events-service.cfapps.<SUBACCOUNT_REGION>.hana.ondemand.com
  btp_saas_registry_api: https://saas-manager.cfapps.<SUBACCOUNT_REGION>.hana.ondemand.com
  uaa_url: https://<SUBACCOUNT_SUBDOMAIN>.authentication.<SUBACCOUNT_REGION>.hana.ondemand.com
  global_account: <GLOBAL_ACCOUNT>
  subaccount_id: <SUBACCOUNT_ID>
  client_id: {{process.env.CIS_LOCAL_CLIENT_ID}}
  client_secret: {{process.env.CIS_LOCAL_CLIENT_SECRET}}
  access_token:
  environment_instance_id:
  app_name:
  app_plan:
}
```

trial-xsuaa

```
vars {
  xsuaa_api_url: https://api.authentication.<SUBACCOUNT_REGION>.hana.ondemand.com
  uaa_url: https://<SUBACCOUNT_SUBDOMAIN>.authentication.<SUBACCOUNT_REGION>.hana.ondemand.com
  subaccount_id: <SUBACCOUNT_ID>
  client_id: {{process.env.XSUA_CLIENT_ID}}
  client_secret: {{process.env.XSUA_CLIENT_SECRET}}
  access_token:
}
```

trial-uas

```
vars {
  uas_api: https://uas-reporting.cfapps.<CENTRAL_REGION>.hana.ondemand.com
  uaa_url: https://<GLOBAL_ACCOUNT>.authentication.<CENTRAL_REGION>.hana.ondemand.com
  subaccount_id: <SUBACCOUNT_ID>
  month: 8
  year: 2026
  client_id: {{process.env.UAS_CLIENT_ID}}
  client_secret: {{process.env.UAS_CLIENT_SECRET}}
  access_token:
}
```

trial-sm

```
vars {
  sm_url: https://service-manager.cfapps.<SUBACCOUNT_REGION>.hana.ondemand.com
  uaa_url: https://<SUBACCOUNT_SUBDOMAIN>.authentication.<SUBACCOUNT_REGION>.hana.ondemand.com
  client_id: {{process.env.SM_CLIENT_ID}}
  client_secret: {{process.env.SM_CLIENT_SECRET}}
  access_token:
}
```

trial-ias (no access_token var — Bruno caches it)

```
vars {
  ias_url: https://<IAS_HOST>
  client_id: {{process.env.IAS_CLIENT_ID}}
  client_secret: {{process.env.IAS_CLIENT_SECRET}}
  redirect_uri: http://localhost:8686/callback
}
```

Step 7 — Verify

Run from the collection root. For each env, fetch a token **first**, then run the folder(s) — the token must exist before the bearer requests execute:

```
# trial-global
bru run auth/get-token.bru --env trial-global
bru run global-accounts/ global-entitlements/ --env trial-global

# trial-local
bru run auth/get-token.bru --env trial-local
bru run local-provisioning/ local-events/ --env trial-local

# trial-xsuaa
bru run auth/get-token.bru --env trial-xsuaa
bru run xsuaa/ --env trial-xsuaa

# trial-uas
bru run auth/get-token.bru --env trial-uas
bru run usage/ --env trial-uas

# trial-sm
bru run auth/get-token.bru --env trial-sm
bru run service-manager/ --env trial-sm
```

Expect **200** everywhere **except**:

- `usage/get-monthly-usage` and `usage/get-subaccount-usage` → **500** (no consumption data in trial — Gotcha #5).
- The IAS `ias/` requests that need OAuth2 `authorization_code` (`get-token-auth-code` , `get-userinfo`) run **only in Bruno Desktop** — click **Get Access Token** in the Auth tab. The no-auth IAS requests (`get-jwks` , `get-openid-configuration`) and `introspect-token` work from the CLI.

Update the Status table as you go.

Gotchas & Design Notes

1. **Region rule.** Accounts + Entitlements + Usage Reporting always use the **central region** (`eu10`); Provisioning + Events + SaaS + Service Manager + XSUAA use the **subaccount region**. This is why `trial-local` mixes `<CENTRAL_REGION>` and `<SUBACCOUNT_REGION>` hosts.
2. **cis-local needs `grantType: clientCredentials`.** Without the create-time parameter the token has only `uas.resource` scope → **403 insufficient_scope** on every provisioning/events call.
3. **Bruno deletes empty Secrets.** That's why secrets live in `.env` and the env files reference them as normal `vars` via `{{process.env.*}}` — not `vars:secret`. This keeps secrets out of the committed files while staying readable by the `bru` CLI. (Desktop-only alternative: Security section.)
4. **IAS: disable the system browser.** Bruno Desktop → Preferences → **Use system browser for OAuth** → **OFF**. The embedded browser is required to intercept the `localhost:8686/callback` redirect.
5. **Usage endpoints 500 in trial.** `monthlyUsage` and `subaccountUsage` have no data in a trial account — the 500 is expected; the requests are kept for completeness. (Note: `subaccountUsage` needs `subaccount_id` in the `trial-uas` env, or it returns 400 instead.)
6. **Token lifetime ~12 h.** On `401`, re-run `auth/get-token`. The IAS token is cached by Bruno for the session; after a Bruno restart, click **Get Access Token** again.
7. **Secrets live only in `.env`; `.gitignore` keeps it out of git.** The env files are secret-free (they reference `{{process.env.*}}`). Bruno writes the runtime `access_token` back into the env files, so still treat `environments/` as sensitive or blank the tokens after a session. Do **not** recreate the stray `json` file from the reference collection (a `bru run` results artifact that leaks a token; normal runs don't emit it).
8. **Bruno OSS = max 2 workspaces.** Unlimited requires the Pro/Ulimate plan (<https://www.usebruno.com/pricing>).

Security

- **Secrets live only in `.env`** at the collection root, which is **git-ignored**. The environment files reference them via `{{process.env.*}}` and are secret-free — safe to commit or share.
- Bruno writes the runtime `access_token` back into the env files during a run. Blank those out (or keep `environments/` untracked) after a session, since a live token is still sensitive.
- The CIS bindings and the XSUAA credential are **write-capable**. For a read-only collection, create the XSUAA credential with `--read-only` and prefer the `cis central-viewer` / `local-viewer` plans.

- **Most secure alternative (Bruno Desktop only):** define credentials as `vars:secret` — Bruno stores them in the OS keystore and never writes them to disk at all. Trade-off: the `bru` CLI cannot read Secrets, so CLI verification (Step 7) won't work. The `.env` approach is the middle ground that keeps CLI compatibility.